Домашнее задание 5 — RL для вызова функций

Модель: Qwen/Qwen3.5-2B (фиксированная ревизия); обучающие данные: Team-ACE/ToolACE; алгоритм: GRPO с LoRA. Seed: 42.

1. Данные и EDA

ToolACE: 11300 диалогов в исходном наборе; обработаны максимум 2500. Выбрано 2280 train и 249 validation примеров, без вызовов функций — 463. Медиана длины промпта: 689 токенов; медиана доступных инструментов: 3. Пропущены: {'unparseable_call': 59, 'no_schema': 156, 'too_long': 7}.

Из каждого диалога извлекаются запросы, списки инструментов и эталонные вызовы. Примеры с некорректно разбираемыми вызовами пропускаются. Диалоги делятся 90/10 до разворачивания, чтобы части одного диалога не пересекались между train и validation. История после вызова инструмента сбрасывается; длинные запросы отсеиваются.

2. Функция награды

Exact: R=1 при совпадении мультимножеств (название функции и аргументы независимо от порядка вызовов), иначе R=0. Если инструменты не нужны, R=1 за непустой ответ без вызовов. Shaped: R=0.2 + 0.3 × совпадения имён / max(n_pred,n_gold) + 0.5 × полные совпадения / max(n_pred,n_gold); невалидный формат получает 0. Для no-tool случаев используется exact.

Примеры: get_weather(city="Moscow") при идентичном эталоне: exact=1, shaped=1; get_weather(city="Paris") при эталоне Moscow: exact=0, shaped=0.5; отсутствие нужного вызова: 0; корректное отсутствие вызова: 1.

Строгая награда напрямую соответствует целевой проверке. Частичная награда поощряет выбор функции даже при неверных аргументах и потенциально уменьшает разреженность сигнала.

3. Эксперименты и гипотезы

RL1: exact, lr=1e-05, 60 шагов. Гипотеза: Проверить влияние строгой награды на формат и точность.

RL2: shaped, lr=1e-05, 60 шагов. Гипотеза: Проверить, помогает ли частичная награда за выбор инструмента.

RL3: shaped, lr=3e-06, 60 шагов. Гипотеза: Проверить, позволяет ли меньший learning rate сохранить исходные навыки.

Общие параметры (из train.py): LoRA r=8, alpha=16, q/k/v/o projections, bf16, 2 генерации, batch=1, accumulation=4, max answer=192, beta=0, seed=42.

4. Итоговый BFCL

Запуск	Overall	Simple	Multiple	Parallel	Par.+Mult.	Irrel.
base	0.695	0.680	0.793	0.560	0.800	0.640
rl1	0.691	0.680	0.793	0.560	0.800	0.620
rl2	0.689	0.680	0.787	0.560	0.800	0.620
rl3	0.700	0.680	0.800	0.580	0.800	0.640

Лучший RL-эксперимент: RL3, overall=0.700; baseline=0.695; разница +0.53 процентного пункта.

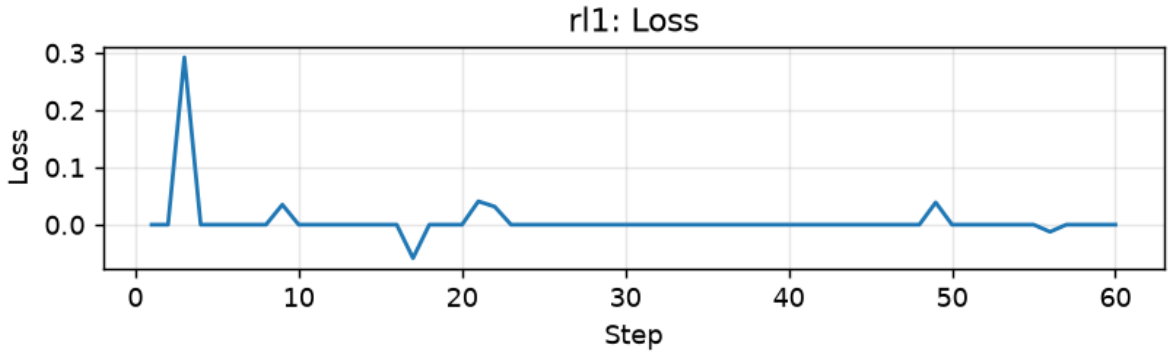
RL1: изменение Overall относительно baseline -0.40 п. п.

RL2: изменение Overall относительно baseline -0.53 п. п.

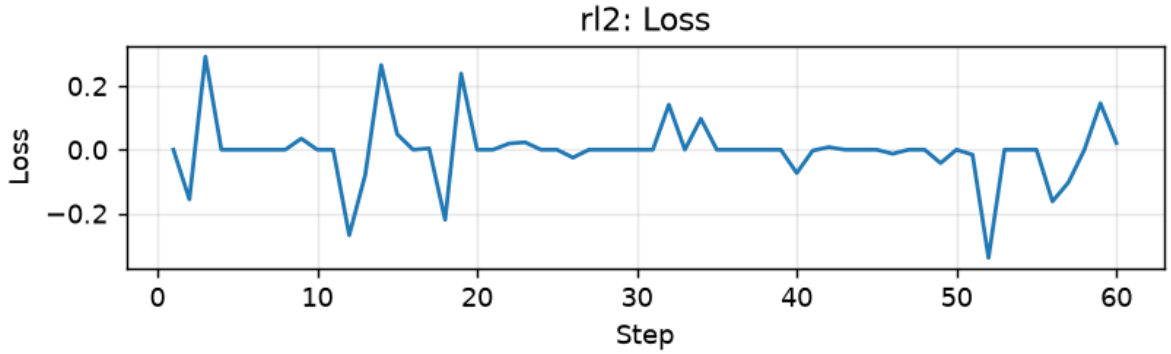
RL3: изменение Overall относительно baseline +0.53 п. п.

На 350 примерах небольшая разница не доказывает устойчивого улучшения; для статистического вывода нужны повторные запуски или дополнительная проверка.

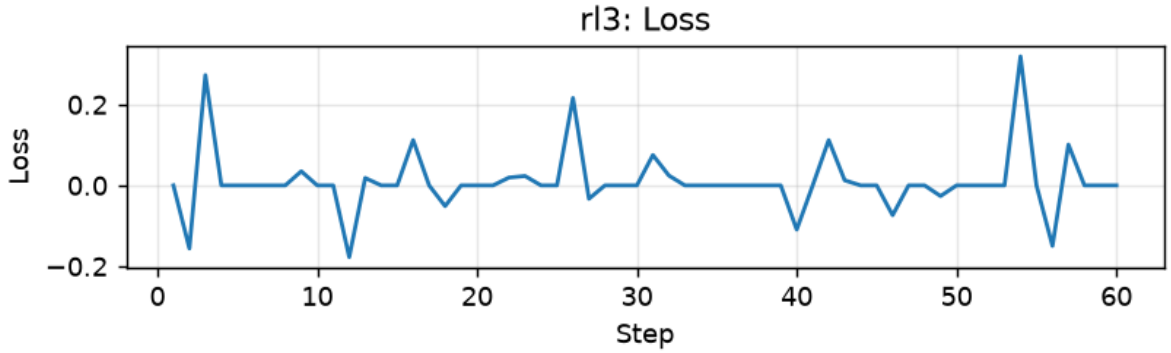
5. Кривые обучения













6. Обсуждение результатов

При одинаковом learning rate переход от exact (RL1) к shaped (RL2) изменил Overall на -0.13 п. п. При одинаковой shaped-награде снижение learning rate (RL2 → RL3) изменило Overall на +1.07 п. п. Эти наблюдения относятся к данным запускам, а не к гарантированному эффекту гиперпараметров.

Рост train reward не гарантирует улучшения BFCL: ToolACE и BFCL различаются по примерам и способу проверки. При одинаковой награде у всех генераций одной группы относительные преимущества GRPO равны нулю, поэтому такая группа не даёт полезного градиентного сигнала.

Simple: 0.680 → 0.680 (+0.00 п. п.).

Multiple: 0.793 → 0.800 (+0.67 п. п.).

Parallel: 0.560 → 0.580 (+2.00 п. п.).

Parallel Multiple: 0.800 → 0.800 (+0.00 п. п.).

Irrelevance: 0.640 → 0.640 (+0.00 п. п.).

7. Анализ ошибок лучшего RL-чекпоинта

1. simple_241 (simple): Ожидался вызов инструмента.

Предсказанные вызовы: []

Эталон BFCL (возможные значения аргументов): [{"US_President_During_Event": {"country": ["USA", ""], "event": ["Civil War"]}}]

Текст ответа модели: The U.S. president during the Civil War was **Abraham Lincoln**. He served from March 1861 until his assassination in April 1865.

2. multiple_119 (multiple): ["Invalid value for parameter 'value': 25. Expected one of ['25']."]

Предсказанные вызовы: [{"database.query": {"conditions": [{"field": "age", "operation": ">", "value": 25}, {"field": "job", "operation": "=", "value": "engineer"}], "table": "user"}}]

Эталон BFCL (возможные значения аргументов): [{"database.query": {"conditions": [[{"field": ["age"], "operation": [">"], "value": ["25"]}, {"field": ["job"], "operation": ["="], "value": ["engineer"]}]], "table": ["user"]}}]

Текст ответа модели: <tool_call> <function=database.query> <parameter=table> user </parameter> <parameter=conditions> [{"field": "age", "operation": ">", "value": 25}, {"field": "job", "operation": "=", "value": "engineer"}] </parameter> </function> </tool_call>

3. live_multiple_597-158-3 (live_multiple): ["Invalid value for parameter 'number_of_rooms': 1. Expected one of ['', 'dontcare']."]

Предсказанные вызовы: [{"Hotels_4_SearchHotel": {"location": "Toronto, Canada", "number_of_rooms": 1, "star_rating": "4"}}]

Эталон BFCL (возможные значения аргументов): [{"Hotels_4_SearchHotel": {"location": ["Toronto, Canada"], "number_of_rooms": ["", "dontcare"], "smoking_allowed": ["", false], "star_rating": ["4"]}}]

Текст ответа модели: <tool_call> <function=Hotels_4_SearchHotel> <parameter=location> Toronto, Canada </parameter> <parameter=star_rating> 4 </parameter> <parameter=number_of_rooms> 1 </parameter> </function> </tool_call>

4. parallel_31 (parallel): Ожидался вызов инструмента.

Предсказанные вызовы: []

Эталон BFCL (возможные значения аргументов): [{"history_fact.fetch": {"depth": ["", "detailed"], "event": ["Treaty of Paris"], "year": ["", 0]}}, {"history_fact.fetch": {"depth": ["", "detailed"], "event": ["Magna Carta"], "year": ["", 0]}}]

Текст ответа модели: The **Treaty of Paris** and the **Magna Carta** are two pivotal documents in English history that shaped the development of the rule of law and the balance of power between the monarchy and the people. ### The Treaty of Paris (1259) The **Treaty of Paris** was signed in 1259 between King **Henry III of England** and **Philip II of France**. It was

5. parallel_multiple_75 (parallel_multiple): ['Wrong number of functions.']

Предсказанные вызовы: [{"chess_club_details.find": {"city": "Boston", "name": "Knight Gambit"}}, {"chess_club_details.find": {"city": "Philadelphia", "name": "Rook Corner"}}]

Эталон BFCL (возможные значения аргументов): [{"route_planner.calculate_route": {"destination": ["Boston"], "method": ["fastest", ""], "start": ["New York"]}}, {"chess_club_details.find": {"city": ["Boston"], "event": ["null", ""], "name": ["Knight Gambit"]}}, {"route_planner.calculate_route": {"destination": ["Philadelphia"], "method": ["fastest", ""], "start": ["Boston"]}}, {"chess_club_details.find": {"city": ["Philadelphia"], "name": ["Rook Corner"]}}, {"route_planner.calculate_route": {"destination": ["New York"], "method": ["shortest"], "start": ["Philadelphia"]}}]

Текст ответа модели: <tool_call> <function=chess_club_details.find> <parameter=name> Knight Gambit </parameter> <parameter=city> Boston </parameter> </function> </tool_call> <tool_call> <function=chess_club_details.find> <parameter=name> Rook Corner </parameter> <parameter=city> Philadelphia </parameter> </function> </tool_call>